

Arquitectura del Sistema TechStore

CASO: TechStore

Tienda online de productos electrónicos

Productos: smartphones, laptops, tabletas, accesorios

Necesidades: gestión de inventario, transacciones de clientes y datos de usuarios

CLIENTE / USUARIO

- Navega catálogo de productos
- Registra cuenta
- Inicia sesión
- Compra productos
- Realiza pagos

HTTPS / SSL

FRONTEND WEB (Container)

Tecnologías: HTML / CSS / JavaScript

Funciones:

- Interfaz gráfica
- Formularios de registro/login
- Carrito de compras
- Catálogo de productos

API REST / JSON

BACKEND APPLICATION (Container)

Tecnologías: Node.js / Express

Funciones:

- Lógica de negocio
- Validación backend (tipos, rangos, formato)
- Gestión de usuarios e inventario
- Gestión de pagos
- Consultas parametrizadas (anti-SQLi)
- Control de autenticación y roles
- Manejo de transacciones ACID

Verifica login



AUTENTICACIÓN

- Login seguro
- Hash bcrypt / argon2
- Tokens JWT / Sesiones
- Control de roles

Procesa pago



PASARELA DE PAGOS

- Validación de pagos
- Confirmación transacción
- Cifrado SSL/TLS
- Cumplimiento PCI-DSS

SQL parametrizado



BASE DE DATOS SQL

PostgreSQL / MySQL

Justificación:

- ✓ Datos estructurados con relaciones
- ✓ Transacciones ACID (críticas en pagos)
- ✓ Consistencia fuerte de stock e inventario
- ✓ Integridad referencial garantizada



USERS

- id (PK)
- nombre
- email
- password_hash
- rol
- fecha_registro



PRODUCTS

- id (PK)
- nombre
- descripcion
- precio
- stock
- categoria



ORDERS

- id (PK)
- user_id (FK)
- total
- fecha
- estado



ORDER_ITEMS

- id (PK)
- order_id (FK)
- product_id (FK)
- cantidad
- subtotal



JUSTIFICACIÓN: ¿Por qué SQL y no NoSQL?

Requerimiento TechStore → Razón SQL

- Pagos y transacciones → SQL garantiza **ACID**
- Relaciones (User-Order-Product) → SQL es **relacional**
- Stock e inventario → SQL evita anomalías de concurrencia
- Reportes (ventas, top productos) → JOINS nativos
- Datos estructurados → Esquema fijo confiable

NoSQL sería útil solo para logs o búsquedas, pero NO para transacciones financieras.



RIESGOS DE SEGURIDAD IDENTIFICADOS

- SQL Injection** — manipulación de consultas
- Fraude en pagos** — datos de tarjeta robados
- Robo de credenciales** — passwords expuestos
- Man-in-the-Middle (MITM)** — interceptación
- XSS** — inyección de scripts en el frontend
- CSRF** — peticiones falsificadas
- Acceso no autorizado** — escalación de privilegios



MEDIDAS DE SEGURIDAD (mapeadas a riesgos)

- SQL Injection → **Consultas parametrizadas** (prepared statements) + validación backend
- Fraude en pagos → **Pasarela PCI-DSS** + tokenización + verificación 3DS
- Robo de credenciales → **Hash bcrypt/argon2** + salt + política de contraseñas robustas
- MITM → **HTTPS / SSL/TLS** obligatorio en toda la comunicación
- XSS → **Sanitización de inputs** + Content Security Policy (CSP)
- CSRF → **Tokens CSRF** + validación de origen
- Acceso no autorizado → **Autenticación JWT** + control de roles vía **DCL (GRANT/REVOKE)**



INSTRUCCIONES SQL APLICADAS A TECHSTORE

DDL

Data Definition

```
CREATE TABLE products (...)  
ALTER TABLE products ADD descuento  
DROP TABLE tmp_carrito
```

Crea estructura: tablas, índices, claves

DML

Data Manipulation

```
INSERT INTO orders (user_id, total)  
UPDATE products SET stock = stock-1  
DELETE FROM carrito WHERE user_id=?  
SELECT * FROM products WHERE cat=?
```

Manipula datos: registrar compra, actualizar stock

DCL

Data Control

```
GRANT SELECT ON products TO admin  
REVOKE DELETE ON users FROM clerk
```

Control de acceso: define qué rol puede Leer/modificar cada tabla

TCL

Transaction Control

```
BEGIN TRANSACTION  
COMMIT — confirma cambios  
ROLLBACK — revierte si falla
```

Garantiza ACID en compras: si falla pago, se revierte el descuento de stock



VALIDACIÓN DE TRANSACCIONES — Ejemplo: Compra de un Smartphone

```
BEGIN TRANSACTION;  
-- 1 Verificar stock disponible (consulta parametrizada)  
SELECT stock FROM products WHERE id = ?;  
-- 2 Descontar stock del producto  
UPDATE products SET stock = stock - 1 WHERE id = ? AND stock > 0;  
-- 3 Registrar la orden  
INSERT INTO orders (user_id, total, fecha) VALUES (?, ?, NOW());  
-- 4 Procesar pago vía pasarela (PCI-DSS)  
IF pago_exitoso THEN  
  COMMIT; -- Confirma compra y descuento de stock  
ELSE  
  ROLLBACK; -- ❌ Revierte TODOS los cambios (stock, orden)  
END IF;
```